REVON: LOG AND MERKLE DIFFERENCING FOR DURABLE STRUCTURED DATA VERSIONING

BY ATHARVA SHEERSH PANDEY[1,a], ADHYAN JAIN[1,b], AND POORNIMA NEDUNCHEZHIAN[1,c]

[1] SCHOOL OF COMPUTER SCIENCE AND ENGINEERING, VIT VELLORE

[a] atharva.sheersh2024@vitstudent.ac.in; [b] adhyan.jain2024@vitstudent.ac.in
[c] CORRESPONDING AUTHOR: poornima.n@vit.ac.in

Revon is a Python prototype for durable, linear-history key/value versioning. We study the choice between log aggregation and hash-pruned comparison over the same SQLite-backed fixed-depth Merkle trie. The contribution is a controlled evaluation of this choice and its workflow costs, rather than a new index or general-purpose database. Our author-run Linux evaluation of the JSON-repaired implementation comprises 516 executions and a separate 72-execution geometry/telemetry supplement. Eight of nine paired hybrid/Merkle timing intervals favour hybrid, while the dense case includes parity. A frozen threshold nevertheless selects a slower path in a separate control. The Windows evaluation comprises 93 cases, each run in a fresh worker process, with disjoint calibration and validation histories, controlled sensitivity experiments, Wine Quality and CO2 public sources, and repeated runs at 100,000 and 1,000,000 records. Its largest threshold candidate wins calibration and uses log throughout the tested intervals; this does not establish a benefit from switching or an optimal crossover. Published CO2 revisions provide observed record changes, while wine corrections remain generated. We report algorithm behaviour, CLI execution phases and whole-worker resources separately. All measured outputs and historical states are checked against semantic oracles. The results describe bounded versioning workflows on the recorded machines; they establish neither storage-engine superiority nor independent-party reproduction.

1. Introduction. A store that versions structured data needs to recover exact earlier states and compare them efficiently, even when updates affect only a few records. Simply archiving files does not provide all of these capabilities. Git links immutable, content-named objects through parent commits, offering a useful starting point [10]. Those objects represent files and directories. Record stores face a related problem: keys may remain in use while update density changes, and a comparison may cover just a few edits or many commits.

We investigate how two correct diff paths behave when they share the same fixed-depth Merkle hash trie, addressed operation history and durable representation. Path copying, cryptographic content addresses and SQLite transactions are established mechanisms. Our contribution is their explicit integration into an auditable comparison of selection policies, with matched semantic outputs and documented measurement boundaries. The scope is limited to the measured versioning operations and workloads, without implying lower costs for structured-data management generally.

Keywords and phrases: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology.

Dolt combines SQL with a commit graph and Prolly Trees [13, 28]. ForkBase contributes POS-Tree/SIRI storage and fork semantics [32], while Noms describes history-independent chunked collections [3]. Revon offers a narrower linear-history key/value prototype. We do not claim feature parity, a new family of persistent indexes, or a generally faster storage engine. The current comparison uses Revon's in-process API and Dolt's CLI; a separate phase breakdown makes the remaining interface costs explicit.

1.1. Research Questions. RQ1: What latency, storage and resource trade-offs are observed for complete versioning workflows, and how do API/CLI boundaries limit cross-system interpretation?

RQ2: How well does an operation-count selector carry from calibration to unseen generated histories when compared with both forced log and forced Merkle on the same representation?

RQ3: How do controlled changes in workload structure, payload and fixed-trie geometry affect differencing costs on the measured histories?

1.2. Contributions. Implementation contribution: Revon integrates established content addressing, path copying and atomic SQLite persistence while exposing both diff paths under one encoded-value contract. This integration is an engineering contribution, not a claim to invent these mechanisms.

Policy contribution: we specify and test an interpretable operation-count rule against its forced-path controls. The calibration choice is frozen before held-out evaluation; adverse choices and the possibility that always-log suffices are retained.

Evidence contribution: observed diff outputs and historical states are checked against separately constructed oracles. Executed sources, raw measurements, failures, environment records and data preparation are retained, including the JSON repair and telemetry limitations.

Evaluation contribution: the author-run Linux campaigns are analysed separately from the 93-case Windows evaluation with disjoint seeds, matched sensitivity controls, Wine Quality and CO2 public sources and repeated scale observations. No timing rows are pooled across these campaigns.

Boundary of the contribution: the experiments concern single-writer linear histories and bounded workloads. They do not establish an optimal selector, a general geometry optimum, query-engine superiority or reproduction by an independent team.

2. Related Work.

2.1. Content Addressing and Persistent Structures. A root digest in a Merkle tree authenticates the larger collection beneath it [22]. Content addressing appears in Git's software histories and in IPFS's networked content [10, 5]. Persistent data structures retain earlier states through path copying [14]. For dictionaries stored on disk, later designs include ordered stratified B-trees [31] and fully persistent B-trees [8]. Revon instead routes hashed keys through a fixed-depth trie. This allows hashes to prune a comparison, but sacrifices ordered range access. Scientific array systems take a different approach: SciDB and TimeArr use layouts and deltas suited to versioned arrays [27, 30]. Version access is also supported within a main-memory database by TardisDB, and replicated versioned structures for wide-area systems are studied by Saquib et al. [25, 26]. These designs help situate Revon's choices. They do not compare its log and Merkle paths on a common history.

Merkle Search Trees use search-tree order and Merkle authentication to reconcile states [4]. Hashes also prune comparisons in Revon, but its routing is fixed-depth and its linear history is centralized, unlike a replicated CRDT. In LBFS, content-defined chunking identifies redundant content [23]. Dolt and Noms use related methods to choose boundaries. Such boundaries can improve deduplication when data shifts; ordered indexes, in turn, permit range operations. We evaluate neither of these capabilities in the Revon benchmark.

2.2. Structured Dataset Versioning. Dataset versioning must account for collaboration alongside the cost of storing and retrieving states. DataHub examines collaborative histories at scale [6]. Within a relational engine, Decibel supports branching and compares version-first, tuple-first and hybrid layouts [20]. OrpheusDB uses partitioning to improve version retrieval when versioning is added to an existing DBMS [19]. The storage/reconstruction trade-off is formalized by Bhattacherjee et al. [7]. Delta Lake builds ACID tables using transaction logs and checkpoints [2], while more recent work selects materialized versions and deltas on a version graph subject to storage and retrieval constraints [18]. Consistent identification and description of releases is the concern of the FAIR-compliant proposal [16]. It complements work on storage but does not specify a diff algorithm. Revon's key/value prototype covers a narrower problem and does not address these broader requirements of dataset management.

2.3. System Design Gap. Noms stores structured values in a Merkle DAG, with support for diff and synchronization [3]. In ForkBase, content addressing is combined with fork and deduplication semantics [32]. Dolt offers ordered indexing resembling a B-tree, structural sharing and native diff through content-addressed Prolly Trees [13, 28]. Git4Data supports snapshot, branch, diff and merge through SQL over a relational engine [17]. The manuscript available for that system is an arXiv preprint, so its reported results are not treated here as peer-reviewed or independently reproduced evidence. Revon's design is more limited. It uses fixed-depth hash routing in a single-writer key/value store with linear history, without SQL database capabilities or a branching system.

The systems and studies closest to the implemented design are summarized in Table 1.

TABLE 1
Related-work summary

System or study	Representation or focus	Relevance to Revon
Persistent DS [14]	Path copying	Formal structural-sharing basis
Git [10]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [5]	Merkle DAG	General content-addressed versioning
MST [4]	Merkle search tree	Authenticated ordered reconciliation
DataHub [6]	Version graph	Collaborative dataset motivation
Decibel [20]	Relational layouts	Native database branching
OrpheusDB [19]	Partitioned versions	Storage/retrieval optimization
Delta Lake [2]	Log/checkpoints	ACID time travel at data-lake scale
Noms [3]	Merkle DAG	Structured content addressing
ForkBase [32]	Forkable CAS	Fork and deduplication semantics
DoltHub [13, 28]	Prolly Trees	Mature production comparator

2.4. Design Implications and Research Positioning. Retaining history, creating versions and later reconstructing or comparing them each impose costs. Snapshots make retrieval straightforward by copying even the unchanged records. Logs postpone work until replay or

checkpointing. Path copying retains the unchanged structure and creates only the paths that changed [14, 7]. Physical layout influences version creation and retrieval in Decibel and OrpheusDB [20, 19], while graph-based analyses express the storage/retrieval trade-off directly [18]. Reproducibility studies address the preservation of builds and artifacts for inspection [1]. Preserving them does not demonstrate that a storage algorithm runs faster, so we report import, commit, checkout, diff and storage separately.

The meaning of a change depends on the research question. Differential dataflow updates query results incrementally as timestamped inputs change [21]. Data provenance explains why and where output facts originated [9]; DSLog records fine-grained array lineage and supports queries over it [33]. Explain-Da-V explains semantic dataset changes through data transformations [29]. Baseline's operation-based differencing accommodates refactorings and schema changes [15]. Revon's output identifies keys that were added, removed or modified between two states. It does not explain causes, transformation lineage, query effects or schema intent. Although these studies help position the work, their outputs and workloads do not permit direct timing comparisons with Revon's key/value diffs.

We compare two correct diff paths over the same durable history, choosing a path from properties of the requested interval. Every Revon-H commit retains a Merkle root and an addressed changeset, so selection leaves version semantics unchanged. Our hypothesis is that log aggregation works well for short, sparse intervals, whereas hash pruning may be useful after longer sequences of operations. Whether the paths cross in performance requires measurement; the switching rule alone cannot establish it. Git4Data illustrates continuing work on database-native version control [17]. Our contribution is confined to adaptive differencing over a fixed-depth hash trie, with Snapshot and Log-only serving as representation baselines. Revon is not a general-purpose data-versioning platform. The retained Linux public-record extension uses generated updates to one TLC release. The Windows evaluation uses generated wine corrections and an observed CO2 publication sequence. These bounded sources do not establish performance across diverse domains, independent hardware, concurrent workloads or production systems.

2.5. Capabilities and comparison boundaries. Table 2 compares system abstractions and the evidence actually used here. A documented capability is not a benchmark result, and a system not evaluated here is not thereby unsupported. The cited ForkBase and Noms APIs describe typed collections; this study does not evaluate a relational layer over them. Git4Data is treated as a preprint about a different abstraction level [17].

TABLE 2
System capabilities and evidence scope; documented features are not performance results

System	Representation / history	Diff / persistence	Branches / queries	Reproducibility and use here
Revon	Fixed-depth hash trie; linear history	Log or Merkle; SQLite objects	No named branches/merge; key/value API	Executed snapshots, semantic proofs and raw trials; measured here
Dolt [13, 28]	Prolly Trees; commit graph	Native version diff; persistent block store	Branches/merge; SQL	Official docs/source; CLI workflow measured, engine not isolated
ForkBase [32]	POS-Tree/SIRI; forkable versions	Hash-pruned diff/merge; addressed chunks	Fork on demand/conflict; typed KV APIs	Primary paper and APIs; no experiment run here
Noms [3]	History-independent chunks; Prolly collections	Hash-based diff/sync/merge; chunk store	Typed collection API; not benchmarked here	Technical overview/source; conceptual comparison
Git4Data [17]	MatrixOne immutable objects/MVCC; table versions	Database-native diff/merge	Snapshot/tag/branch; SQL extensions	Preprint/BranchBench claims belong to its authors; not run here

3. Revon Design. The architecture and the scope implemented in this study are shown in Figure 1.

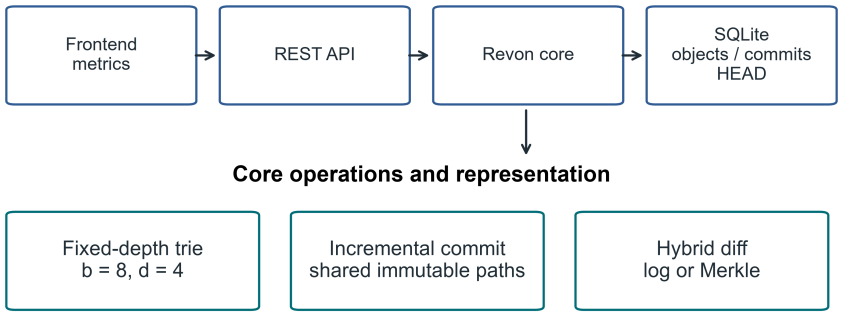


FIG 1. Revon architecture and the implemented research scope

3.1. Addressed Object Model. Trie nodes and changesets are serialized as canonical JSON, with each object's SHA-256 digest serving as its name. The commit digest includes the root, parent, changeset, message and timestamp. Readable labels such as v1 and v2 are aliases and have no effect on object identity. The current commit is identified by a durable HEAD reference.

3.2. Fixed-Depth Merkle Hash Trie. The main experiment uses b=8 and d=4. At each level, routing takes three bits from SHA-256(key); twelve bits select among 4,096 possible leaf buckets. Because a bucket retains the full keys, a shared routing prefix does not make two keys indistinguishable. With uniform hashes, expected bucket occupancy is N/b^d. We evaluate other b,d pairs separately and make no assumption that the default is optimal.

3.3. Incremental Hashing and Commit. Revon first canonicalizes a mutation batch and routes its keys, grouping updates that share a trie prefix. It rewrites each affected leaf once and rebuilds the ancestors on those paths. Unchanged child hashes are carried into the new internal nodes, allowing the unchanged structure to be shared. One SQLite transaction writes the resulting objects and changeset, commit metadata, version alias and measurements, and updates HEAD. When the repository is reopened, Revon verifies stored payload hashes and references.

3.4. Hybrid Diff. The log path walks the ancestry of the requested interval and combines operations for each key. The Merkle path compares the roots recursively: equal hashes end that part of the search, while differing paths lead to examination of their entries. Before choosing a path, Revon-H validates both version identifiers, checks ancestry in each direction and counts the changeset operations. An available ancestor path with a count no greater than T is eligible for log aggregation. Otherwise, Revon-H uses Merkle comparison. For a reverse request on the log path, it exchanges old and new values. A version compared with itself gives an empty diff. Forced log rejects unrelated versions, and an unknown version raises an error. Revon-M always requests the Merkle path, although its public API for a version pair still checks ancestry. Both paths return the same sorted semantic diff; commit and checkout are separate operations.

We retain T=4,096 in the main Linux workflow matrix, as specified in the original workflow protocol. A separate Linux control tests the earlier frozen candidate, T=16,384. The threshold counts operations, whereas b^d specifies bucket capacity. We tune neither parameter from the Linux results.

3.5. JSON value ownership and identity. Revon accepts JSON nulls, booleans, finite numbers, strings, lists and objects whose keys are strings. It validates and detaches input containers before retaining them, and returns detached containers through read and diagnostic views. Immutable scalars may be shared. Before any write, it rejects Python-only values that are unsupported, non-string keys in nested objects, non-finite numbers and cyclic containers. Value equality follows the existing v1 JSON encoding, which uses sorted object keys, compact separators and UTF-8, with NaN excluded. The encodings of true, 1, 1.0 and signed floating zero therefore stay distinct. These rules define the Python encoding contract; they do not assert conformance to a cross-language canonical-JSON standard.

Algorithm 1 (batch update). Validate the inputs and take ownership of their values. Before interning, reject overlapping puts/deletes and deletions of missing keys. Read the old values and keep only changes to encoded identity. Group these effective mutations by routing prefix, then rewrite each touched leaf, sort its entries and hash the complete payload. Rebuild the affected ancestors while reusing unchanged child hashes, and record the addressed changeset and commit. The ownership invariant prevents callers from changing retained values through earlier inputs or supported reads.

Algorithm 2 (log composition). Traverse the ancestor interval chronologically. For each key, keep the existence/value at its first old endpoint and at its last new endpoint. Drop that key only if both endpoints agree in existence and encoded value. Thus, inserting and then deleting a key cancels, as does updating and then restoring its value; a null value is still distinct from an absent key. Sort the remaining keys. A reverse request exchanges the endpoints and swaps the added/deleted labels.

Algorithm 3 (selection). Validate the versions and check ancestry in either direction. Add the changeset operation counts across the interval. Choose log for an eligible interval whose count is at most T, and Merkle otherwise. Merkle comparison skips addressed subtrees with equal hashes and compares values in unequal leaves. Both paths use the same

encoded-identity rule and return detached outputs.

Cost model. Let N denote the record count, d the depth, b the branching factor, m the keys in a batch, h the interval length, p the accumulated operations, u the distinct interval keys and s the serialized output bytes. Expected leaf occupancy under uniform routing is N/(b^d), with N as the worst case. Processing a batch involves validating and copying inputs, performing m routed lookups of old values and scanning the occupied leaves. Touched leaves must be sorted and serialized, and affected internal child maps must be copied. The work therefore cannot be bounded by a dataset-independent O(d).

Log composition includes ancestry traversal and O(p + u log u) work, together with encoded-value comparisons and output copying/serialization. For Merkle comparison, the work depends on the node pairs visited, sorting child slots and the leaf entries examined, as well as comparing values and sorting the output. Both public version-pair modes traverse ancestry. A reverse request may initially walk to genesis in the direction that fails. Producing the output costs at least an amount proportional to s. Larger JSON values introduce comparison and copying costs beyond those characterized by scalar-only benchmarks.

Historical nodes, changesets and commits are loaded eagerly from SQLite into memory. Although the repository stores its objects durably, they are resident while Revon runs, and the retained history must fit in memory. The amount of retained payload determines the work of opening and verifying a repository. No disk-page cache or eviction policy is provided. The atomicity of SQLite persistence does not demonstrate the production concurrency or operational guarantees that we have not tested.

Figure 2 traces the decision made for a version pair, showing the eligibility checks and the treatment of reverse requests.

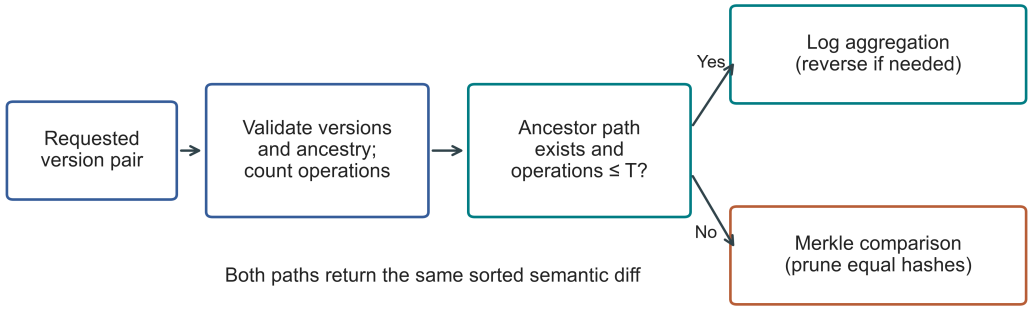


FIG 2. Actual hybrid selection for a version pair; T denotes the operation threshold. Invalid versions raise an error before selection. Both paths share the persistent trie

4. Experimental Methodology.

4.1. Environment and evidence boundaries. The retained primary campaign ran on a separate Ubuntu n2-standard-4 cloud VM; Table 3 identifies its environment and two source snapshots. The Windows campaign ran on a separate author-operated host. Differences in hardware, OS, runtime and storage remain confounded, and neither campaign is an independent-party reproduction.

TABLE 3
Linux environment and separately retained evidence

Property	Recorded configuration
Machine	GCP n2-standard-4; Intel Cascade Lake; 4 vCPUs; 16 GB RAM; us-east1-c
Operating system	Ubuntu 24.04.5 LTS; x86_64; kernel 7.0.0-1011-gcp
Storage	100 GB balanced persistent disk; no swap configured
Runtime	CPython 3.14.3, Clang 22.1.1 build; GIL enabled; JIT not enabled; SQLite 3.50.4
Comparator and packages	Dolt 2.3.1; psutil 7.1.0; PyArrow 25.0.1
Primary source	linux-validation-20261008: archived repaired source, fixed before measurement
Supplement source	linux-final-supplement-20261008: same core; zero-aware I/O sampler and geometry history checks
Background conditions	Google Ops Agent active; filesystem cache not cleared; monitoring included

Each campaign retains source hashes, a frozen protocol, environment records, correctness proofs and raw observations. The Linux preflight gates passed 67 and 69 tests; the Windows preflight gate passed 77. Packaging failures and the public-data premeasurement exclusion remain documented. Detailed records and commands are described in Appendices A and B.

4.2. Variants and workloads. The variants retain history in different ways. Snapshot stores a complete state at every version; Log-only stores the starting state and subsequent operations. Revon-M requests Merkle diff, whereas Revon-H chooses its diff path. Dolt uses a primary-key SQL table and returns native JSON diff. For Dolt, SQL INSERT and CSV table import are timed separately, giving six workflow variants across five systems. Their later operations follow the same semantic contract, although their interfaces and storage features differ. The figures use the workload identifiers assigned in Table 4.

TABLE 4
Linux workloads; all primary histories have ten generated commits

ID / scenario	Rows	Changes / commit	Locality
S1 Small sparse	1,000	10	Spread
S2 Medium sparse	10,000	10	Spread
S3 Medium dense	10,000	1,000	Spread
S4 Large sparse	100,000	100	Spread
S5 Application-key local	100,000	100	Lexically adjacent keys
S6 Range local	100,000	100	Contiguous key intervals
S7 Repeated key	100,000	100	Repeated small key set
S8 Hash-route local	100,000	100	Shared 9-bit hash prefix
P1 TLC public records	100,000	100	Generated fare corrections

Eight synthetic scenarios, each with six variants, two warm-ups and seven measured trials, account for 432 executions. The public-record cases contribute 27 executions across Revon-M, Revon-H and Dolt SQL, and the two threshold controls contribute another 54. Three million-row diagnostics bring the campaign to 516 executions: 114 warm-ups, 399 measured trials and three diagnostics. All 516 finished with verified outputs. Failed timing executions are not silently replaced or omitted.

4.3. Public dataset and generated histories. The public-data extension uses NYC TLC's January 2024 yellow-taxi release [24]. A checksum fixes the Parquet input, which has 2,964,624 rows. With seed 20261001, we use a dispersed affine permutation to obtain nested subsets of 10,000, 100,000 and 1,000,000 rows. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Each key is formed from the release, the file-checksum prefix and the physical row ordinal. It identifies a row within this fixed file, not the same trip in different releases. Values, including nulls, are serialized as compact UTF-8 JSON with sorted keys. In the million-row subset, 47,277 passenger counts are missing, and serialization succeeds for every row. Identical field values do not cause rows to be combined. The affine permutation used for sampling is not uniformly random.

We generate the public edit histories with ten commits. On the 100,000-row subset in the Linux campaign, a commit makes 100 corrections; in the million-row diagnostic, it makes 1,000. A correction adds 0.01 to both fare and total, leaving the other retained fields untouched. All systems receive identical records and mutations. These edits are constructed for the experiment and are not observed longitudinal changes. The public-data builder ignores the inherited payload_bytes=8 field in the configuration. All eleven fields remain in a public value, and payload size is accounted for through the measured serialized bytes. The harness stores a canonical record as a JSON-encoded string in the common value column. We therefore do not measure native mutation of nested objects or queries on individual fields.

4.4. Timing, correctness and uncertainty. The workflow is preparation, repository construction, timed operations, semantic verification and analysis. Initial import, commit, diff and checkout have separate boundaries. Diff outputs use the same sorted key and old/new-value-hash contract. Table 5 separates algorithm work from interface and worker costs; correctness checks and decision probes are outside operation timers.

TABLE 5
Timing and interpretation boundaries

Quantity	Revon / same-representation control	Dolt CLI workflow	Interpretation
Diff latency	Version API, selected path and canonical output	Native JSON-diff command, parsing and canonical output	Complete interface contract; no engine-only ranking
Phase probe	Separate ancestry/count probe; includes detached-view copying	Command, JSON parse and output-construction timers	Neither probe nor command isolates pure tree-engine work
Correctness	All states, reverse/identity, integrity and reopen outside timers	Observed native rows and historical checkout outside timers	Observed values compared with an independently constructed oracle
Resources	Worker plus sampled children, including oracle/setup	Worker plus sampled Dolt processes	10 ms samples; short processes/peaks may be missed
Persistent bytes	Closed local files; old Linux conditioning retained	Repository directory files	Includes metadata/indexes; not isolated algorithm overhead

For each execution, we compare the observed endpoint diff and every historical state with independent workload oracles. Revon also checks reverse/identity requests, stored-object integrity and reopen behaviour. Audits reconstruct the plan and semantic hashes; they verify internal consistency, not a fresh timing experiment.

The historical Linux estimates use seven fresh-repository trials and paired percentile-bootstrap timing intervals for fixed histories. The Windows campaign records seven cached queries within each fresh repository and reports independent-history or repository medians separately. These experimental units are not interchangeable; neither estimates a population of machines.

Resource sampling covers the worker and observed descendants, including setup and verification. It is not product-only memory or operation-only CPU. The primary Linux read-counter ambiguity is retained; the Linux supplement and Windows sampler distinguish supported zero counters from missing counters. Windows process I/O bytes are not direct physical-disk traffic.

Repository footprint measures persistent files, including metadata and representation costs. The Linux measurement is taken before closing/compacting; the Windows measurement follows close and the specified verification steps. These differently conditioned values are not pooled or interpreted as pure tree overhead.

4.5. Frozen threshold and geometry controls. An existing Windows calibration/held-out study selected the largest candidate, T=16,384; it did not identify a crossover. The Linux threshold controls preserve that choice. The Windows campaign uses a separately frozen design, source snapshot and disjoint seeds, described below; its threshold results are analysed separately.

The retained Linux controls compare forced log, forced Merkle and T=16,384 hybrid at 8,192 and 32,768 operations over 10,000 rows. The Windows measurements do not retune these controls.

The Linux geometry supplement evaluates five configurations on one fixed spread workload. The local sensitivity experiment adds three generated histories and matched equal-capacity geometries. Bucket capacity and operation threshold remain distinct parameters; neither study establishes a universal geometry optimum.

4.6. Separate local calibration and validation. The local campaign fixes N=12,000, H=8, payload=32 bytes and b=8,d=4 for the primary selector comparison. Two calibration seeds and three different validation seeds cross spread/repeated-key histories with 64, 512 and 4,096 changes per commit. Forced log, forced Merkle and T=512/4,096/16,384/65,536 hybrid share a durable representation within each case. Two warm-ups precede seven measured query blocks; configuration order is deterministically shuffled. Each case starts in a fresh worker, while queries within it share resident objects and filesystem cache. No cold-cache condition is claimed.

Calibration minimises the equally weighted mean of hybrid case medians normalised by the lower forced-path median; exact ties choose the smaller T. The selected parameter and UTC time are written before the first validation worker starts. Validation outcomes do not select a new winner. Sensitivity changes rows, history interval, payload, geometry and routing in a prespecified staged design. At fixed accumulated operations, interval length and changes per commit trade off; the design is not a full factorial. Appendices and retained protocols give all seeds and matched controls.

4.7. Additional public histories and repeated scale. Wine Quality contributes 6,497 red/white records, keyed by colour and source row ordinal [11]. Eight batches of 64 seeded quality-field corrections are generated for each of three histories; they are not observed specimen revisions. The CO2 package supplies published NOAA-derived CSV revisions [12]. We retain the eleven most recent file-changing snapshots acquired before timing and exclude one header-only publication failure before freezing the experiment. Ten populated

snapshots give nine observed transitions, initially 812 records. The source contains unnamed trailing columns, which are preserved as additional fields. Dates identify records; CSV row order and formatting are discarded. The result is a published repository history, not NOAA transaction logs or atmospheric-science evidence.

Three CO2 repositories repeat the same observed history, whereas wine and scale use distinct generated seeds. Each source is measured through Revon-M, T=4,096 Revon-H and native Dolt JSON diff, rotating model order across runs. The scale study holds H=4, C=64 and payload=32 fixed at 100,000 and 1,000,000 records, with three seeds per size/system. Medians and ranges of repository medians remain descriptive. Source URLs, licences, versions, byte checksums and transformation rules accompany the data; a bounded publication history was chosen instead of bulk Software Heritage acquisition on this host.

The complete count matrix is retained in Table 17 in Appendix B; exact public-data checksums and the original Linux public protocol are in Table 16 there. The main text retains timing boundaries and experimental units needed to interpret the results.

5. Linux Results.

5.1. Hybrid versus forced Merkle. Revon-M and Revon-H store the same persistent representation and changesets. The ablation therefore changes the requested diff strategy. Figure 3 and Table 6 report all nine comparisons with repeated trials. Eight hybrid intervals exclude parity, while the interval for dense S3 includes one. The sparse and repeated-key cases show the benefit of access to log aggregation under these conditions. They do not demonstrate that switching is universally advantageous or that it saves storage at the representation level.

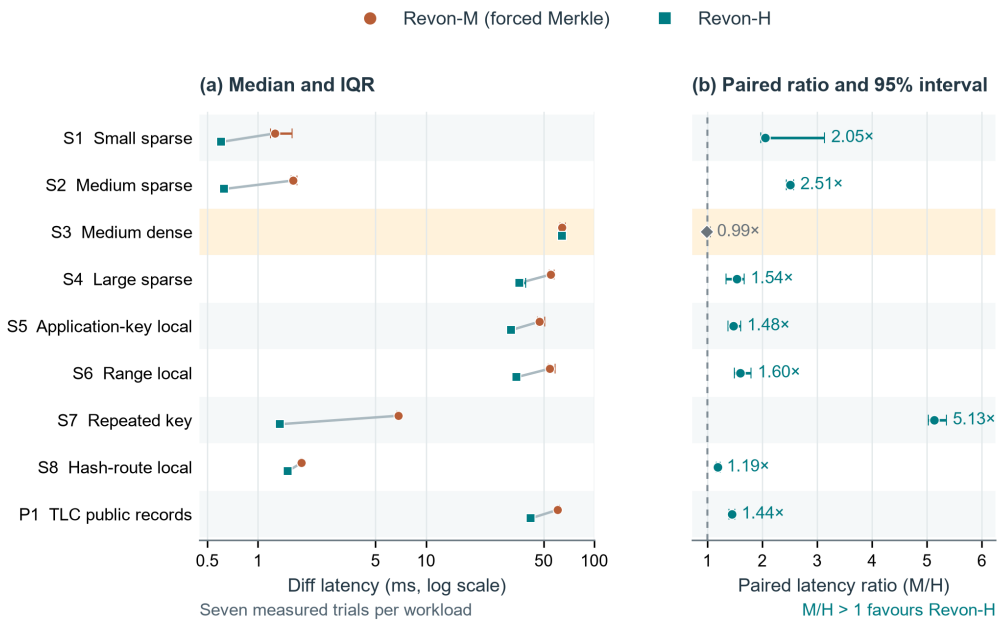


FIG 3. Linux Revon-M and Revon-H diff comparison. (a) Median latency with interquartile ranges on a logarithmic axis; paired medians are connected within each workload. (b) Median matched-trial M/H latency ratio with paired percentile bootstrap 95% intervals (20,000 resamples); the dashed line denotes parity. Each workload has seven measured trials. The highlighted S3 dense case has an interval containing parity; the other eight favour H. Ratios are medians of matched-trial ratios, not ratios of the two latency medians. Workload definitions are in Table 4. These intervals concern repeated timings for fixed histories on one VM, not independent datasets or machines.

TABLE 6
Paired M/H diff ablation; milliseconds and median trial ratio [95% interval]

ID	M (ms)	H (ms)	M/H ratio [interval]	H path
S1	1.259	0.604	2.05 [1.97, 3.14]	log
S2	1.620	0.629	2.51 [2.44, 2.57]	log
S3	64.135	64.183	0.99 [0.97, 1.06]	merkle
S4	54.897	35.749	1.54 [1.34, 1.67]	log
S5	47.253	31.967	1.48 [1.38, 1.60]	log
S6	54.352	34.537	1.60 [1.50, 1.80]	log
S7	6.862	1.345	5.13 [5.02, 5.35]	log
S8	1.817	1.503	1.19 [1.17, 1.23]	log
P1	60.597	41.964	1.44 [1.39, 1.50]	log

In P1, the median diff latency is 41.964 ms for Revon-H and 60.597 ms for forced Merkle. Both process the same eleven-field public records with the same generated updates. S3 reaches 10,000 accumulated operations, causing H to use Merkle. Its near-parity result is consistent with the variants sharing a diff path, with selection work added for H.

5.2. Threshold control and adverse result. Both threshold workloads appear in Figure 4 and Table 7, including the case that contradicts the frozen policy's choice. H uses log at 8,192 operations. At 32,768 operations it switches to Merkle, despite forced log being faster in each of the seven matched measured trials. Operation count by itself therefore fails to

select the fastest path in these cases. The Linux control fulfils the validation task, but the optimal threshold remains unresolved.

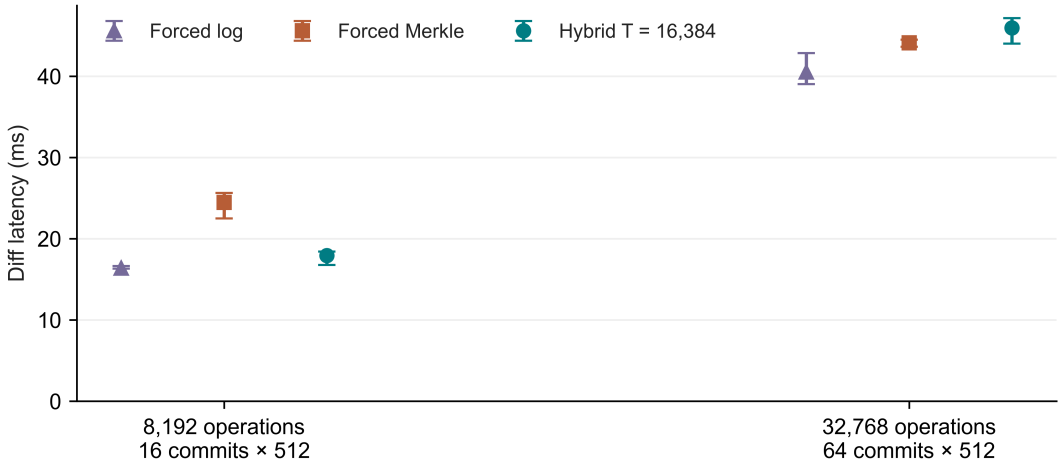


Fig 4. Linux control for the previously frozen T=16,384 policy; medians and IQRs from seven measured trials per path, with different history lengths labelled explicitly

Table 7
Frozen threshold controls; diff medians in milliseconds

Operations / history	Forced log	Forced Merkle	Hybrid	Selected path
8,192 / H=16	16.364	24.466	17.926	log
32,768 / H=64	40.520	44.120	45.944	merkle

5.3. Complete workflow comparison. Table 8 compares the canonical diff outputs of the Revon-H API and Dolt SQL/CLI workflows. Each of the nine paired Dolt/H intervals lies above one. This finding applies to the complete workflows. The measurements combine data-structure costs with process startup, language, serialization, query execution and resident state. Figure 5 presents commit and checkout separately, preventing the diff result from being interpreted as an advantage for every operation.

Table 8
Dolt SQL/CLI versus Revon-H diff; milliseconds and paired ratio [95% interval]

ID	Dolt (ms)	H (ms)	Dolt/H ratio [interval]
S1	90.704	0.604	148.15 [139.77, 160.15]
S2	95.515	0.629	148.70 [140.08, 150.80]
S3	297.617	64.183	4.70 [4.54, 4.77]
S4	152.472	35.749	4.25 [3.84, 4.34]
S5	125.594	31.967	3.73 [3.71, 3.87]
S6	135.765	34.537	3.88 [3.69, 4.12]
S7	97.451	1.345	71.73 [68.95, 74.32]
S8	110.966	1.503	71.24 [68.83, 77.06]
P1	168.576	41.964	4.02 [3.77, 4.07]

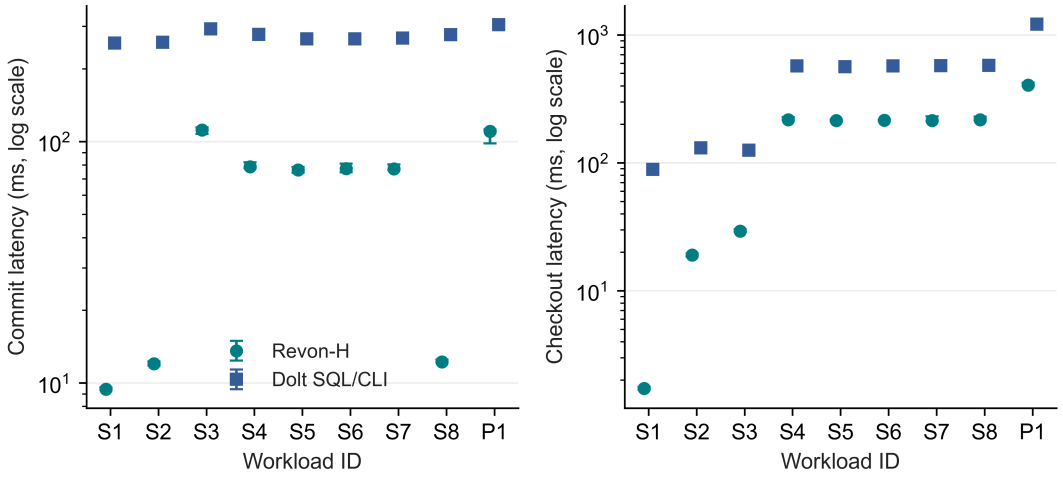


FIG 5. Linux incremental-commit and full-checkout latency for Revon-H and Dolt SQL/CLI; points show medians and bars show IQRs of seven trials, using logarithmic latency

5.4. Storage and memory costs. Figure 6 and Table 9 place the latency results alongside repository footprint and RSS for the complete worker. Addressed changesets are retained by both M and H. Their small measured directory-size difference arises from the workflows: taking the log path does not discard trie data. Dolt uses different serialization, compression and indexing as well. A fast diff can therefore come with a much larger repository. The reported resource costs are part of the assessment of efficiency.

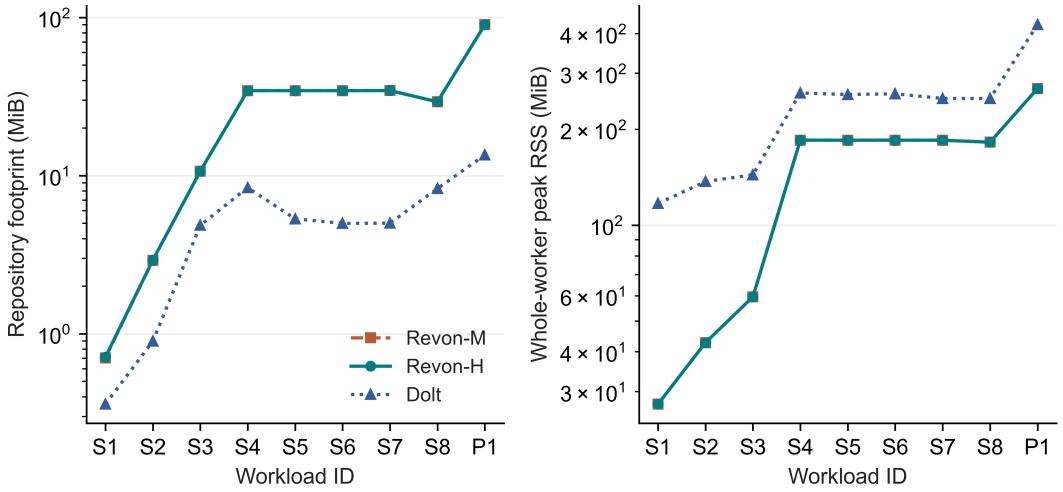


FIG 6. Median repository directory size before compaction and median whole-worker peak RSS across seven Linux trials; logarithmic scales, including shared workload and verification memory in RSS

TABLE 9
Repository footprint and worker RSS; medians in MiB

ID	M disk	H disk	Dolt disk	H RSS	Dolt RSS
S1	0.7	0.7	0.4	27.4	116.9
S2	2.9	2.9	0.9	42.7	137.1
S3	10.7	10.7	4.8	59.5	143.9
S4	34.4	34.4	8.4	184.9	260.1
S5	34.4	34.4	5.3	184.8	257.2
S6	34.4	34.4	5.0	184.9	258.6
S7	34.5	34.5	5.0	184.8	250.0
S8	29.3	29.3	8.3	182.2	249.9
P1	90.1	90.0	13.5	268.6	426.5

5.5. Million-record feasibility. In the retained Linux million-row diagnostic, we create and verify one fresh repository for each of M, H and Dolt. Verification checks all eleven historical states and the common diff oracle. Table 10 reports the individual observations, with no IQRs, confidence intervals or performance ranking. Because the interval contains 10,000 accumulated operations, exceeding T=4,096, H uses Merkle. Each run demonstrates completion of this bounded workload on the VM. One run per system cannot establish reliable relative scaling or identify an optimal configuration; the repeated Windows scale runs are reported separately in Section 6.4.

TABLE 10
Single-run million-record diagnostics; n=1 per system, no comparative inference

System	Import (s)	Commit (ms)	Diff (ms)	Checkout (s)	RSS (GiB)
Dolt	24.902	547.311	817.034	11.454	2.555
Revon-H	32.289	2961.901	1233.601	4.729	2.561
Revon-M	32.454	2979.717	1265.367	4.685	2.561

5.6. Fixed-trie geometry and telemetry. Figure 7 and Table 11 report the geometry supplement. Three configurations, b4-d6, b8-d4 and b16-d3, each have 4,096 leaf buckets. Their expected occupancy is consequently equal, and comparing them examines the effect of routing shape. When b=8 is held fixed, changing depth changes bucket capacity as well as the balance between traversal and leaf scanning. These observations come from one workload on one VM and cannot identify a geometry that is optimal in general.

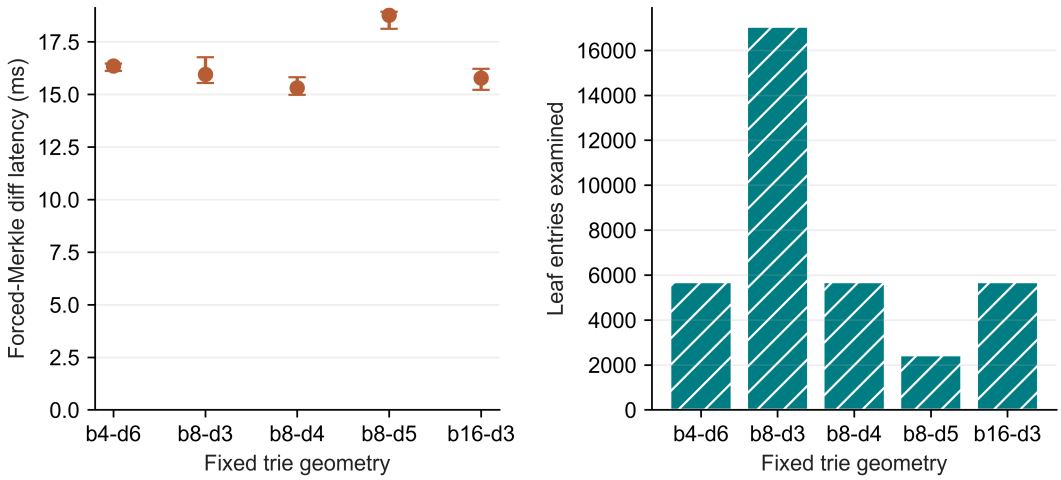


FIG 7. Linux fixed-trie geometry supplement; left shows forced-Merkle diff medians and IQRs from seven trials, right shows median examined leaf entries

TABLE 11
Geometry work and footprint; one fixed spread workload

Geometry	Buckets	Diff (ms)	Leaf entries	Node pairs	Disk (MiB)
b4-d6	4096	16.355	5682	3628	4.32
b8-d3	512	15.949	17048	585	6.88
b8-d4	4096	15.316	5682	3729	4.25
b8-d5	32768	18.765	2426	6089	6.18
b16-d3	4096	15.775	5682	3929	8.12

We checked the telemetry regression with real isolated workers, testing both valid all-zero I/O samples and unavailable counters. Both tests passed. The subsequent Linux supplement consists of 27 executions on a fixed 10,000-row workload: M, H and Dolt each have two warm-ups and seven measured trials. The recorded rows preserve zero read bytes as a measured value rather than leaving them blank. This confirms how the counters are handled. It does not establish cold-storage behaviour or eliminate sampling loss, and it leaves the primary latency summaries unchanged.

5.7. Representation baselines. For the representative S4 spread workload, Table 12 gives descriptive medians for all six workflows. The complete eight-scenario synthetic matrix is retained in the GitHub reproducibility archive (Appendix A), along with trial dispersion, normalized storage and the separate SQL/CSV import variants. In the common harness, Snapshot and Log-only control for the choice of representation, without offering the same production features. Their results show the cost of keeping complete states or replaying operations, to be considered alongside the costs measured for Revon.

TABLE 12
Representative S4 workflow medians; complete eight-scenario matrix in the GitHub reproducibility archive

Workflow	Import (s)	Commit (ms)	Diff (ms)	Checkout (ms)	Disk (MiB)
Snapshot	0.071	69.577	624.860	168.158	53.5
Log-only	0.072	2.474	16.483	159.538	5.0
Revon-M	1.488	78.842	54.897	215.661	34.4
Revon-H	1.644	78.590	35.749	217.482	34.4
Dolt SQL	1.663	277.593	152.472	572.585	8.4
Dolt CSV	1.335	273.425	149.325	559.342	7.9

6. Windows Evaluation Results.

6.1. Held-out selector behaviour. Calibration selected T=65,536, the largest candidate. Its normalised calibration score was 1.014. On 18 held-out histories, its mean normalised latency was 1.012, versus 1.101 for T=4,096. Agreement with the lower forced-path median was 18/18 and 12/18 respectively. These are descriptive choices relative to noisy forced-path medians, not classification accuracy against a known optimal strategy. Table 13 and Figure 8 report all profile groups rather than selecting only favourable cases.

All tested intervals contain at most 32,768 operations, so T=65,536 requests log throughout this matrix. Thus the calibration winner is behaviourally an always-log policy over these intervals. Its selection cannot establish that adaptive switching is beneficial or locate a global crossover. Hybrid/forced ratios below one may reflect timing variation despite equivalent selected paths. The earlier adverse Linux result is retained unchanged. Production defaults are not altered by this experimental selection.

TABLE 13
Held-out median of three history medians; fixed N/H/payload and no population interval

Locality	Operations	Forced log ms	Forced Merkle ms	H4096 ms	Calibrated ms
spread	512	1.913	4.622	1.907	1.837
spread	4,096	25.714	38.878	26.550	27.312
spread	32,768	92.792	104.71	106.56	91.775
repeated-key	512	0.564	1.661	0.549	0.573
repeated-key	4,096	5.608	14.342	6.311	4.664
repeated-key	32,768	62.290	74.366	77.092	67.721

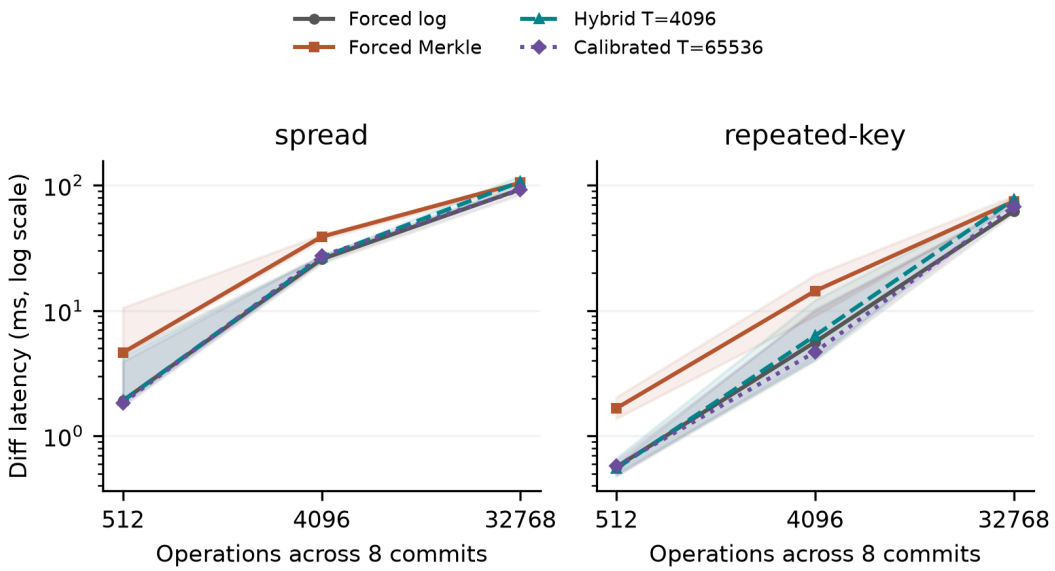


FIG 8. Held-out selector comparison at N=12,000 and H=8. Lines show medians across three generated-history medians; shaded bands span their minimum and maximum. Each history has seven cached measured queries. Bands are descriptive ranges, not 95% confidence intervals. The calibrated threshold uses log at every shown operation count. Forced paths share the same durable representation.

6.2. Sensitivity and resource interpretation. The recorded decision probe is outside diff timing and reads a detached public changeset view, so its copying work is not pure selector overhead. A separate post-campaign diagnostic extracts the unchanged production function prefix through input validation, ancestry and internal operation counting, stopping before diff dispatch. Across the 18 held-out histories and two thresholds, 252,000 measured warm prefix calls give a median of case/threshold medians of 2.674 microseconds (range 1.611-3.678). The extracted decision agrees with production dispatch. This microbenchmark excludes diff traversal and detached-view copying; it does not isolate cold-cache overhead or provide a causal amount to subtract. The original 93-case records are unchanged.

The 21 sensitivity cases change one prespecified factor or matched control across three seeds. Payload=256 bytes, N=24,000, fixed-operation H=16, equal-capacity b4-d6/b16-d3 geometries, and a matched C=8 hash-route/spread pair are all retained in the selector summary. Geometry cases match the base history state hashes; only routing geometry changes. The fixed-operation interval comparison trades H against changes/commit and does not isolate both independently. These results concern the tested profiles and do not select a generally best geometry.

Forced-Merkle medians across the three history medians were rows24000: 47.857 ms; history16: 40.122 ms; payload256: 39.950 ms; geometry4x6: 43.004 ms; geometry16x3: 38.500 ms; route8: 0.174 ms; route-control8: 0.903 ms. Full per-history timing, chosen paths and resource scopes remain available in the retained summaries. These exploratory comparisons receive no multiple-testing significance claim.

The three sampler off/on pairs gave T=4,096 median-latency ratios ranging from 0.950 to 1.206. Off precedes on within each pair, and background/cache conditions are uncontrolled; this diagnostic does not identify a causal overhead correction and is not subtracted from other timings. Resource counters span whole workers. Tests reject the replacement of observed zero I/O by missing data, but Windows counters still cannot identify database logical reads or direct physical-disk traffic.

6.3. Public data and interface phases.

TABLE 14
Additional public sources: median and range of three repository medians; CO2 repeats one history

Source / system	Runs	Diff ms	Range ms	Import s	Persistent MiB
co2/revon-m	3	0.612	0.562-0.661	0.033	1.191
co2/revon-h	3	0.309	0.302-0.318	0.035	1.184
co2/dolt	3	256.98	251.87-262.89	0.634	0.178
wine/revon-m	3	4.281	4.260-4.964	0.189	10.832
wine/revon-h	3	2.247	1.956-2.366	0.191	10.812
wine/dolt	3	296.76	246.75-302.81	0.819	1.360

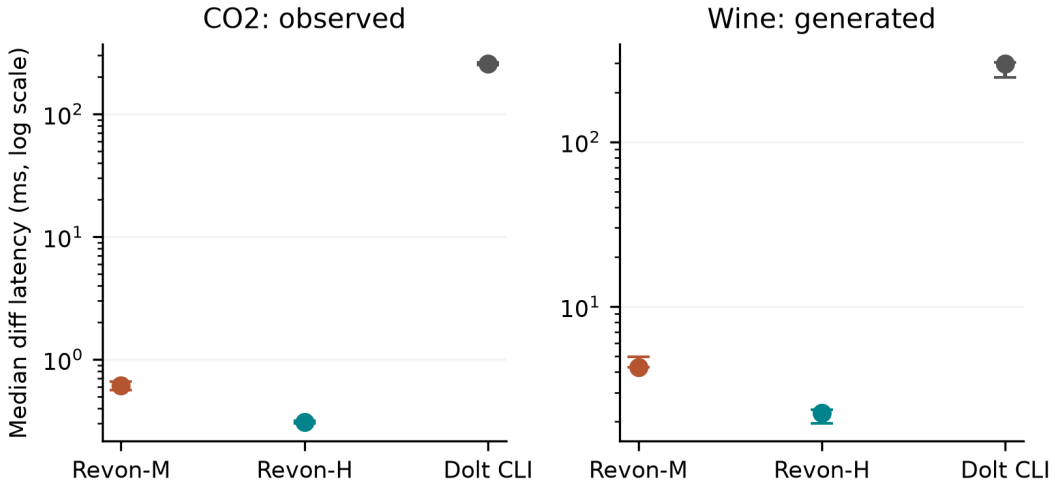


FIG 9. Additional public data. Points show medians of three repository medians and bars show their full range. CO2 repeats the same observed publication history; wine uses three generated correction histories. Each repository contributes seven cached queries. The API/CLI interfaces differ; these are workflow latencies, not isolated storage-engine measurements.

For co2-observed, the Dolt command phase had a median of 256.063 ms, JSON interpretation 0.501 ms and common output construction 0.503 ms. The command accounts for 99.6% of the median complete diff latency, while combining process startup, native work and native JSON emission. The decomposition cannot assign that whole interval to a storage engine, nor justify subtracting a separately measured startup constant.

For wine-generated, the Dolt command phase had a median of 288.464 ms, JSON interpretation 4.445 ms and common output construction 3.258 ms. The command accounts for 97.2% of the median complete diff latency, while combining process startup, native work and native JSON emission. The decomposition cannot assign that whole interval to a storage engine, nor justify subtracting a separately measured startup constant.

6.4. Repeated local scale observations.

TABLE 15
Repeated Windows scale study: medians of three seed/repository summaries; full ranges in evidence

Rows / system	Import s	Commit ms	Diff ms	Checkout s	Worker RSS GiB
100,000/revon-m	1.116	54.309	5.649	0.209	0.248
100,000/revon-h	1.109	53.044	1.004	0.173	0.242
100,000/dolt	1.662	759.99	283.65	0.662	0.223
1,000,000/revon-m	10.614	88.233	33.694	1.862	1.939
1,000,000/revon-h	10.738	85.362	1.013	1.575	1.925
1,000,000/dolt	11.281	890.97	320.30	3.706	0.976

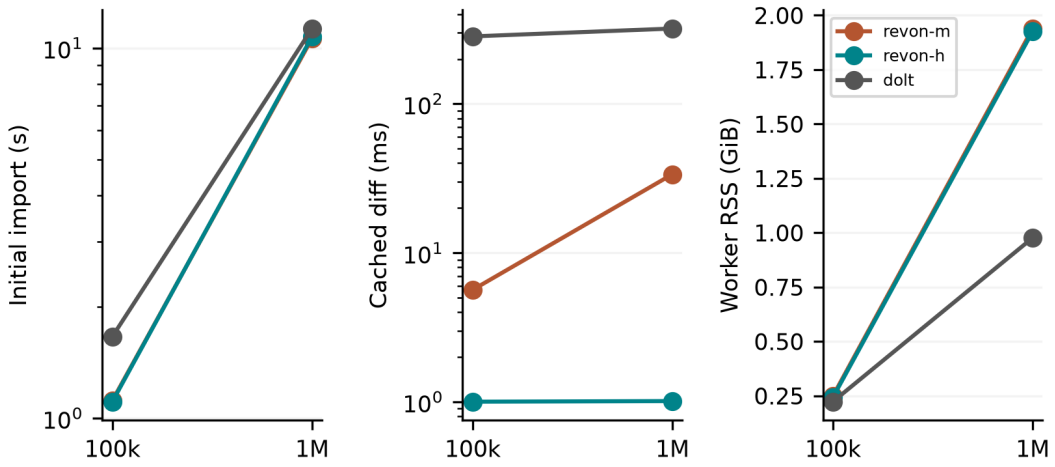


FIG 10. Local scale observations with three generated histories and fresh repositories per size/system. Curves connect descriptive medians at two sizes, not fitted asymptotic scaling laws. Diff queries are cached; import and checkout have separate boundaries. Sampled peak RSS includes workload/oracle generation, storage work, verification and sampled children. The Linux million-row n=1 diagnostics remain a different campaign.

The repeated Windows scale runs are analysed separately from the Linux diagnostics. The three generated histories share the same initial synthetic distribution and differ in updates; they are not three independent public datasets. Memory availability and background processes are recorded in the manifest, not controlled away. These observations support conditional descriptions of the measured workflows, not a population-level ranking or a general scaling law.

6.5. Key takeaways. RQ1: the workflow measurements expose latency, persistence and worker-resource trade-offs, with API/CLI costs still preventing an engine-only ranking. RQ2: some fixed histories benefit from log relative to forced Merkle, but the Windows calibration winner uses log everywhere tested; optimal switching remains unestablished. RQ3: controlled sensitivity histories make payload, routing and geometry effects visible within the tested design, without identifying a universally best layout. Observed CO2 transitions broaden revision realism, while generated wine and synthetic histories retain their own scope limits.

7. Discussion. RQ1 concerns the complete versioning contract. The Linux and local experiments both retain import, commit, diff, checkout and persistent footprint, but their rows are analysed separately. Snapshot and Log-only expose representation costs; the M/H/log controls hold the Revon representation fixed. Dolt provides a richer SQL/versioning interface. Its command-phase decomposition improves transparency without making its native tree cost comparable to an isolated Python tree call.

RQ2 concerns transfer to unseen histories, not merely repeated timing of one history. The Windows split controls primary N, H, payload and geometry and freezes selection before validation. Its largest candidate behaves as always-log within the measured interval range. The adverse Linux selection and dense-case parity remain useful counterexamples. A more complex selector would require new calibration and untouched testing; these results do not justify adding complexity or changing the production threshold.

RQ3 concerns controlled changes to the fixed trie and its workload. Equal-capacity geometries can differ in traversal depth and ancestor reconstruction while having equal expected occupancy under uniform routing. Payload encoding, touched leaves and output materialisation also contribute. The Windows sensitivity set uses independent seeds and matched controls alongside the Linux study of one fixed history, but does not estimate every factor interaction or a population optimum.

The JSON repair is included in every current Linux experiment. Previously, mutable aliases could remain connected to stored values. Python equality could also treat true as equal to 1, or 1 as equal to 1.0. We repaired those semantics by detaching inputs and outputs and comparing v1 encoded identities, keeping the object encoding and on-disk schema intact. Archived earlier timings describe the implementation before this repair. The Linux trials exercise the repaired source on the supplied histories, and unit tests cover null/absence, nested ownership, unsupported values and rollback.

8. Limitations And Future Work. External validity: Linux results come from one author-run cloud VM, and Windows results from one author-run host. Both campaigns were conducted by the authors; the Windows campaign does not reproduce the Linux environment. Hardware, OS, runtime, virtualization and storage vary together between campaigns. Background services and filesystem caching remain active; the local manifest records limited available RAM and process conditions. No entirely idle, cold-cache or OS-causal claim follows.

Workload validity: the original TLC histories remain generated from one monthly release, with n=1 Linux scale diagnostics. The Windows evaluation includes three held-out generated seeds per primary profile, generated wine corrections and one observed CO2 publication sequence. Three CO2 repositories repeat that sequence rather than independently sampling its history. A header-only upstream revision is excluded before timing, and unnamed columns are retained without scientific interpretation. Repeated synthetic scale histories broaden observations but share their initial distribution. More domains, observed histories and hardware are still needed for broad generalisation.

Implementation scope: Revon uses a trie of fixed geometry and loads retained nodes, changesets and commits eagerly, without evicting them from memory. It supports a single writer and linear history. We have not evaluated named branches, merge commits, conflict handling, schema evolution, remotes, ordered scans, crash fault injection or concurrent workloads. The use of atomic SQLite transactions does not, by itself, demonstrate all the durability and operational guarantees needed in production.

Measurement and inference: process startup, native work and JSON emission remain combined in Dolt's command interval. Whole-worker RSS includes oracle/history state and is not product-only memory. Ten-millisecond sampling can miss short-lived processes, and

Windows I/O counters do not identify physical-disk or database logical reads. The off/on sampler diagnostic also confounds execution order and cache state. The historical bootstrap intervals concern fixed-history timings; Windows three-history ranges are descriptive and have no multiple-comparison correction. Source and semantic-hash checks establish internal consistency, not independent performance reproduction.

Selection and design: calibration is limited to four thresholds, two generated seeds and the declared primary profiles. T=65,536 is the largest candidate and does not switch within the tested operation range. That outcome leaves a broadly useful adaptive selector unestablished. The staged sensitivity design is not a full factorial; H and density trade off at fixed operation count, and hash-route tests require their own low-density control. A new selector or crossover claim requires a separately frozen experiment.

Possible storage changes include binary nodes, value chunks, packed objects, compression and reachability-based garbage collection. Range operations could be supported through an ordered Merkle index or content-defined chunking. Each proposal needs its own semantic and performance evaluation; its effects cannot be inferred from the fixed-trie results presented here.

9. Conclusion. Revon integrates established fixed-trie content addressing, changesets and atomic SQLite persistence to expose two semantically consistent ways to compute a version diff. The retained Linux evidence comprises 516 executions and a 72-execution supplement, including dense-case uncertainty and an adverse threshold choice. The Windows evaluation comprises 93 worker cases and 3,402 checked queries, with held-out generated histories, Wine Quality and CO2 public sources, interface-phase measurements and repeated scale observations. Calibration selects T=65,536, which behaves as always-log in this matrix and does not establish optimal switching. The combined evaluation covers controlled workload variations and public data while retaining API/CLI, sampling, dataset and hardware limits. The contribution is an auditable prototype and a bounded empirical trade-off study; a new general-purpose database, universal speed advantage and independent-party reproduction are not established.

Funding. No funding was received by the authors for this work.

Competing interests. The authors have no competing interests to declare.

Ethics and consent to participate. This computational study recruited no participants and performed no human or animal intervention. It uses synthetic histories and publicly released de-identified TLC records. We make no claim of institutional ethics approval or exemption.

Consent for publication. No identifiable participant material is included, so consent for publication is not applicable.

Data and code availability. NYC TLC provides the original public taxi source [24]; UCI Wine Quality [11] and the published CO2 package [12] provide the additional sources. Code, frozen experiment snapshots and evidence are available at https://github.com/atharvasheersh/Revon. The Windows reproducibility archive is Revon_Review_Revision_Reproducibility_20261016.zip; the original Linux archive remains available separately. Original TLC records and generated databases are not redistributed. The public-source manifest records licences and transformations; Appendix A describes access.

Acknowledgment. We thank the open-source Dolt project and the researchers whose studies of dataset versioning helped shape our experimental design.

REFERENCES

[1] M. Akhlaghi et al. (2021). Toward long-term and archivable reproducibility, Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, https://doi.org/10.1109/MCSE.2021.3072860. [Online]. Available: https://arxiv.org/abs/2006.03018.

[2] M. Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores, Proc. VLDB Endowment, vol. 13, no. 12, pp. 3411-3424, https://doi.org/10.14778/3415478.3415560.

[3] Attic Labs. Noms Technical Overview. GitHub. [Online]. Available: https://github.com/attic-labs/noms/blob/master/doc/intro.md.

[4] A. Auvolat and F. Taïani (2019). Merkle Search Trees: Efficient State-Based CRDTs in Open Networks, in Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS), pp. 221-230, https://doi.org/10.1109/SRDS47363.2019.00032.

[5] J. Benet (2014). IPFS: Content Addressed, Versioned, P2P File System, arXiv:1407.3561, [Online]. Available: https://arxiv.org/abs/1407.3561.

[6] A. Bhardwaj et al. (2015). DataHub: Collaborative Data Science and Dataset Version Management at Scale, in Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR), Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.

[7] S. Bhattacherjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran (2015). Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff, Proc. VLDB Endowment, vol. 8, no. 12, pp. 1346-1357, https://doi.org/10.14778/2824032.2824035.

[8] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas (2020). Fully persistent B-trees, Theoretical Computer Science, vol. 841, pp. 10-26, https://doi.org/10.1016/j.tcs.2020.06.027. [Online]. Available: https://cs.au.dk/~gerth/papers/tcs20.pdf.

[9] P. Buneman, S. Khanna, and W.-C. Tan (2001). Why and where: A characterization of data provenance, in Database Theory: ICDT Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.

[10] S. Chacon and B. Straub (2014). Git Internals: Git Objects, in Pro Git, 2nd ed., [Online]. Available: https://git-scm.com/book/en/v2/Git-Internals-Git-Objects.

[11] P. Cortez, A. Cerdeira, F. Almeida, T. Matos and J. Reis (2009). Wine Quality [Dataset], UCI Machine Learning Repository, https://doi.org/10.24432/C56S3T. CC BY 4.0.

[12] DataHub Contributors (2026). CO2 PPM - Trends in Atmospheric Carbon Dioxide, NOAA-derived published data package, accessed October https://github.com/datasets/co2-ppm. ODC-PDDL-1.0; acquired revision identifiers and file hashes are retained in the experiment manifest.

[13] DoltHub (2026). Dolt Storage Engine: Block Store, Dolt Documentation, accessed Oct. 2, [Online]. Available: https://www.dolthub.com/docs/architecture/storage-engine/block-store/.

[14] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan (1989). Making data structures persistent, J. Comput. Syst. Sci., vol. 38, no. 1, pp. 86-124, https://doi.org/10.1016/0022-0000(89)90034-2.

[15] J. Edwards and T. Petricek (2025). Baseline: Operation-based evolution and versioning of data, arXiv:2512.09762, preprint, updated 2026. [Online]. Available: https://arxiv.org/abs/2512.09762.

[16] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez (2024). Standardised versioning of datasets: A FAIR-compliant proposal, Scientific Data, vol. 11, art. 358, https://doi.org/10.1038/s41597-024-03153-y. [Online]. Available: https://www.nature.com/articles/s41597-024-03153-y.

[17] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang (2026). Git4Data: Database-native version control for AI agents, arXiv:2609.02106, preprint. [Online]. Available: https://arxiv.org/abs/2609.02106.

[18] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee (2024). To store or not to store: A graph theoretical approach for dataset versioning, in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), pp. 479-493, https://doi.org/10.1109/IPDPS57955.2024.00049. [Online]. Available: https://arxiv.org/abs/2402.11741.

[19] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran (2017). OrpheusDB: Bolt-on Versioning for Relational Databases, Proc. VLDB Endowment, vol. 10, no. 10, pp. 1130-1141, https://doi.org/10.14778/3115404.3115417.

[20] M. Maddox et al. (2016). Decibel: The Relational Dataset Branching System, Proc. VLDB Endowment, vol. 9, no. 9, pp. 624-635, https://doi.org/10.14778/2947618.2947619.

[21] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard (2013). Differential dataflow, in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.

[22] R. C. Merkle (1988). A digital signature based on a conventional encryption function, in Advances in Cryptology: CRYPTO '87, Lecture Notes in Computer Science, vol. 293, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32.

[23] A. Muthitacharoen, B. Chen, and D. Mazières (2001). A low-bandwidth network file system, in Proc. 18th ACM Symp. Operating Syst. Principles (SOSP), pp. 174-187, https://doi.org/10.1145/502034.502052.

[24] New York City Taxi and Limousine Commission (2024). TLC Trip Record Data: Yellow Taxi, January public dataset, accessed Oct. 1, 2026. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[25] N. Saquib, C. Krintz, and R. Wolski (2023). Replicated versioned data structures for wide-area distributed systems, IEEE Transactions on Parallel and Distributed Systems, vol. 34, no. 1, pp. 207-224, https://doi.org/10.1109/TPDS.2022.3217969. [Online]. Available: https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf.

[26] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann (2019). Versioning in main-memory database systems: From MusaeusDB to TardisDB, in Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM), pp. 169-180, https://doi.org/10.1145/3335783.3335792. [Online]. Available: https://www-db.cs.tum.edu/~schuele/data/tardis.pdf.

[27] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker (2012). Efficient versioning for scientific array databases, in Proc. IEEE 28th Int. Conf. Data Engineering (ICDE), pp. 1013-1024, https://doi.org/10.1109/ICDE.2012.102. [Online]. Available: https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f.

[28] T. Sehn (2024). Dolt's Storage Engine, DoltHub Blog, Feb. 29, [Online]. Available: https://www.dolthub.com/blog/2024-02-29-storage-engine/.

[29] R. Shraga and R. J. Miller (2023). Explaining Dataset Changes for Semantic Data Versioning with Explain-Da-V, Proc. VLDB Endow., vol. 16, no. 6, pp. 1587-1600, https://doi.org/10.14778/3583140.3583169.

[30] E. Soroush and M. Balazinska (2013). Time travel in a scientific array database, in Proc. IEEE 29th Int. Conf. Data Engineering (ICDE), pp. 98-109, https://doi.org/10.1109/ICDE.2013.6544817. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.

[31] A. Twigg, A. Byde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie (2011). Stratified B-trees and versioning dictionaries, in Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), [Online]. Available: https://arxiv.org/abs/1103.4282.

[32] S. Wang et al. (2018). ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications, Proc. VLDB Endowment, vol. 11, no. 10, pp. 1137-1150, https://doi.org/10.14778/3231751.3231762.

[33] J. Zhao and S. Krishnan (2024). Compression and in-situ query processing for fine-grained array lineage, arXiv:2405.17701, [Online]. Available: https://arxiv.org/abs/2405.17701.

APPENDIX A: REPRODUCIBILITY

Readers can find the code and experiment records in the Revon GitHub repository. The paper/ directory contains the original Linux reproducibility archive and Revon_Review_Revision_Reproducibility_20261016.zip, which retains the Windows campaign, source snapshots, raw observations, protocols and internal audits. The README files explain dependency setup, public-source acquisition, execution and verification [1]. Repository updates do not replace the frozen source inside an experiment bundle.

The two Linux bundles and the Windows study belong to different campaigns and must not be pooled. Original TLC records and generated repositories are rebuilt using the recorded preparation steps. The local public-source manifest records licences, acquisition versions, checksums and the premeasurement CO2 exclusion. Auditors run on disposable copies with localised data paths, preserving collected bytes. Internal audits do not independently reproduce timing; matching semantic hashes do not guarantee identical elapsed times.

APPENDIX B: METHODS AND EVIDENCE DETAILS

To identify what was run, each bundle contains the executed source and its hashes, environment records, frozen workload definitions, execution order, raw rows and audits. We set PYTHONHASHSEED=20261008 and archived the working revision itself, since repository HEAD alone would not identify it. Before timing on the VM, the primary campaign passed 67 unit tests and the supplement passed 69. The first supplement preflight had failed: the transferred files did not include the test_data directory. We created that empty fixture directory and reran the full test suite before measuring the supplement. The retained records include this packaging failure.

For every primary execution, we start a fresh worker and create a repository. The protocol records model order, which is shuffled deterministically and rotated within repetition blocks. Import timing ends when the first durable version has been created. The reported incremental-commit time is the median of the individual commit timers for that history. A diff runs from the first version to the last and returns sorted changed keys, together with old/new value hashes, as canonical JSON. Checkout writes the complete sorted key/value state in canonical UTF-8 JSON. A Dolt timer includes launching the CLI, parsing JSON and normalizing the output. Revon is called through its in-process API. This difference in timing boundaries prevents a speed claim about the engines alone.

We perform correctness checks outside the operation timers. In every primary execution, we verify the hashes of the initial and final states, check all H+1 historical states, and compare the observed canonical semantic diff with a workload oracle constructed independently. The verification JSON retains observed and expected hashes, rather than just an expected count of changed keys. The post-run audit regenerates workloads and checks the exact matrix, the selected paths and the identity of the source, data and protocol. It also recomputes all summaries. We audit the downloaded evidence locally as well. These procedures check internal integrity; they do not constitute an independent reproduction of the timings.

We exclude warm-ups and report the median and interquartile range (IQR) of the seven measured repository trials. For a latency ratio, we first pair the trials, then report the median ratio and a paired percentile bootstrap interval from 20,000 resamples. Values above one favour the denominator. These exploratory 95% intervals describe repeated timings of a fixed workload on one VM. They are not population confidence intervals, and we apply no multiple-comparison correction. Taking the median of paired ratios need not give the same value as dividing the two latency medians. We retain every completed trial, including timing outliers.

The parent process samples the worker and its descendants every 10 ms, recording RSS, CPU and I/O counters. Peak measurements include workload generation, the oracle state and verification along with the storage work. They cannot be assigned to the storage engine alone. A short peak or a short-lived subprocess's counters can fall between samples. The original sampler treats an all-zero read counter as missing, so we draw no read-I/O conclusion from the primary campaign. In a separate supplement, we distinguish a recorded

zero from an unavailable counter and retain those data apart from the primary results. A cached workload can examine logical data while recording zero physical read bytes.

We measure repository footprint as directory size before closing or compacting it. This includes indexes, metadata and the costs of the representation. Logical payload normalization and selected compaction probes from the first trial are also retained. These measurements describe the workflow rather than isolating tree overhead. CPU, RSS and I/O span the full worker lifecycle, giving them a different scope from the individual operation timers.

The earlier Windows study evaluated thresholds of 1,024, 2,048, 4,096, 8,192 and 16,384 operations on predefined calibration and validation profiles that did not overlap. Minimizing normalized calibration latency selected T=16,384 before validation. Its held-out normalized score was 1.032, against 1.297 for T=4,096; always-log scored 1.000 in both splits. These cached-query measurements belong to their archived source and environment, and do not measure the repaired Linux implementation. Selection of the largest candidate in the search does not identify an optimum. The earlier long-history and three-seed pilots are retained as exploratory supplementary evidence.

In the Linux threshold control, T=16,384 remains frozen. We use 10,000 rows and 512 changes per commit, with H=16 and H=64 giving 8,192 and 32,768 accumulated operations. Forced log, forced Merkle and hybrid are run in separate workers, each with two warm-ups and seven measured trials. We do not change the threshold in response to Linux results. These two cases check how the existing choice carries over to Linux, without constituting a new calibration/validation cycle.

The geometry supplement uses a single spread workload: 10,000 rows, ten commits, 100 changes per commit, 32-byte synthetic values and seed 20260824. Diff is fixed to forced Merkle for b4-d6, b8-d3, b8-d4, b8-d5 and b16-d3. Each configuration runs in a fresh worker for two warm-ups and seven measured trials. We rotate the order to limit confounding between configuration and time. Every execution checks the eleven historical states and endpoint diff outside the timers. Geometry timings follow the same adapter output contract, but the resource peaks measured by its inherited in-process monitor are kept separate from the external RSS measurements.

For the local study, refer to the frozen protocol and full case list in review-revision-20261016/campaign. Calibration seeds 202610161/202610162 are distinct from validation/sensitivity seeds 202610171–202610173. Workers use PYTHONHASHSEED=20261016. The monitor condition uses seed 202610190; public generated corrections use seeds 202610201–202610203; scale uses seeds 202610221–202610223. Queries share a repository within each worker, and every new worker builds fresh persistent state. The chosen threshold is timestamped before validation. There is no post-validation selection or pooled Linux/Windows summary.

The Windows Dolt instrumentation invokes the same native JSON-diff command and parser contract as the baseline adapter, with explicit phase timers. A preflight compares its actual output against the native adapter and the independent semantic oracle. The Windows Revon queries verify observed output hashes, all historical state hashes, reverse/identity requests, object integrity and reopen behaviour outside timing. Failed setup checks and data-quality exclusions precede measurement and remain documented. The exact query matrix and timing source hashes are retained; changed source requires a different campaign.

A portable audit copies evidence to a disposable directory, imports the executed snapshot in a fresh interpreter, redirects only the recorded absolute public-data location, regenerates workloads and state/diff oracles, checks every configuration/trial/order identity and recomputes the numerical summaries. It then confirms that the original collection hashes remain intact. The archive provides these checks and acquisition recipes; they validate

evidence consistency rather than elapsed times or a new research team's reproduction.

TABLE 16
Checksum-pinned public data and Linux protocol

Item	Definition
Source	NYC TLC yellow_tripdata_2024-01.parquet; 2,964,624 source rows
Raw SHA-256	c4d59da7bbc8abaeeeb1727947ee93d9891a71acb42854bd80db1571b2030510
Prepared million-row SHA-256	6608b7680e82432ca22f90a71d4994676ffd290c4fc85f17f91fab03d4c7b661
P1	100,000 records; H=10; C=100; 2 warm-ups + 7 measured trials per system
Scale diagnostic	1,000,000 records; H=10; C=1,000; one trial per system; no interval or ranking

TABLE 17
Windows experiment matrix; cached queries are distinct from independent history or repository repetitions

Component	Fresh workers	Histories / control	Warm-up queries	Measured queries
Calibration	12	2 seeds × 2 localities × 3 densities	144	504
Held-out	18	3 different seeds × same profiles	216	756
Sensitivity	21	3 seeds × 7 matched variations	252	882
Monitor diagnostic	6	3 off/on pairs, one fixed history	72	252
Public sources	18	2 sources × 3 runs × M/H/Dolt	36	126
Scale	18	2 sizes × 3 seeds × M/H/Dolt	36	126
Total	93	Different campaigns are never pooled	756	2646